

LAB REPORT

Switches, Lights and Multiplexers

3220102382 潘谷雨

2024.11.28

1. Problem Description

(1)switch display

The DE2 board provides 18 toggle switches, called SW17–0, that can be used as inputs to a circuit, and 18 red lights, called LEDR17–0, that can be used to display output values. The codes shows a simple VHDL entity that uses these switches and shows their states on the LEDs. Since there are 18 switches and lights it is convenient to represent them as arrays in the VHDL code, as shown. We have used a single assignment statement for all 18 LEDR outputs, which is equivalent to the individual assignments

```
LEDR(17) <= SW(17);
LEDR(16) <= SW(16);
...
LEDR(0) <= SW(0);
```

The DE2 board has hardwired connections between its FPGA chip and the switches and lights. To use SW17–0 and LEDR17–0 it is necessary to include in your Quartus II project the correct pin assignments, which are given in the DE2 User Manual. For example, the manual specifies that SW0 is connected to the FPGA pin N25 and LEDR0 is connected to pin AE23. A good way to make the required pin assignments is to import into the Quartus II software the file called DE2 pin assignments.csv, which is provided on the DE2 System CD and in the University Program section of Altera's web site. The procedure for making pin assignments is described in the tutorial Quartus II Introduction using VHDL Design, which is also available from Altera.

It is important to realize that the pin assignments in the DE2 pin assignments.csv file are useful only if the pin names given in the file are exactly the same as the port names used in your VHDL entity. The file uses the names SW[0] ... SW[17] and LEDR[0] ... LEDR[17] for the switches and lights, which is the reason we used these names in the codes (note that the Quartus II software uses [] square brackets for array elements, while the VHDL syntax uses () round brackets).

LIBRARY ieee;

USE ieee.std_logic_1164.all; -- Simple module that connects the SW switches to the LEDR lights

ENTITY part1 IS

PORT (SW : IN STD_LOGIC_VECTOR(17 DOWNTO 0);

LEDR : OUT STD_LOGIC_VECTOR(17 DOWNTO 0)); -- red LEDs

END part1;

ARCHITECTURE Behavior OF part1 IS

BEGIN

LEDR <=SW;

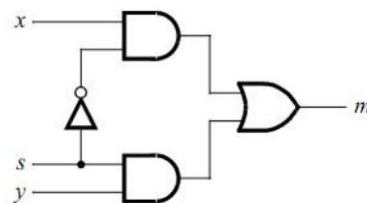
END Behavior

Perform the following steps to implement a circuit corresponding to the code on the DE2 board.

1. Create a new Quartus II project for your circuit. Select Cyclone II EP2C35F672C6 as the target chip, which is the FPGA chip on the Altera DE2 board.
2. Create a VHDL entity for the code and include it in your project.
3. Include in your project the required pin assignments for the DE2 board, as discussed above. Compile the project.
4. Download the compiled circuit into the FPGA chip. Test the functionality of the circuit by toggling the switches and observing the LEDs.

(2)2-to-1 multiplexer

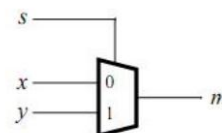
Figure 57a shows a sum-of-products circuit that implements a 2-to-1 multiplexer with a select input s . If $s=0$ the multiplexer's output m is equal to the input x , and if $s=1$ the output is equal to y . Part b of the figure gives a truth table for this multiplexer, and part c shows its circuit symbol.



a) Circuit

s	m
0	x
1	y

b) Truth table



c) Symbol

Figure 57. A 2-to-1 multiplexer.

Perform the following steps to implement a circuit corresponding to the code on the DE2 board.

1. Create a new Quartus II project for your circuit.
2. Include your VHDL file for the eight-bit wide 2-to-1 multiplexer in your project. Use switch SW 17 on the DE2 board as the s input, switches SW7–0 as the X input and SW15–8 as the Y input. Connect the SW switches to the red lights LEDR and connect the output M to the green lights LEDG7–0.
3. Include in your project the required pin assignments for the DE2 board. As discussed in Part I, these assignments ensure that the input ports of your VHDL code will use the pins on the Cyclone II FPGA that are connected to the SW switches, and the output ports of your VHDL code will use the FPGA pins connected to the LEDR and LEDG lights.

4. Compile the project.
5. Download the compiled circuit into the FPGA chip. Test the functionality of the eight-bit wide 2-to-1 multiplexer by toggling the switches and observing the LEDs.

(3) 5-to-1 multiplexer

In Figure 57 we showed a 2-to-1 multiplexer that selects between the two inputs x and y . For this part consider a circuit in which the output m has to be selected from five inputs u , v , w , x , and y . Part a of Figure 58 shows how we can build the required 5-to-1 multiplexer by using four 2-to-1 multiplexers. The circuit uses a 3-bit select input $s_2s_1s_0$ and implements the truth table shown in Figure 59b. A circuit symbol for this multiplexer is given in part c of the figure.

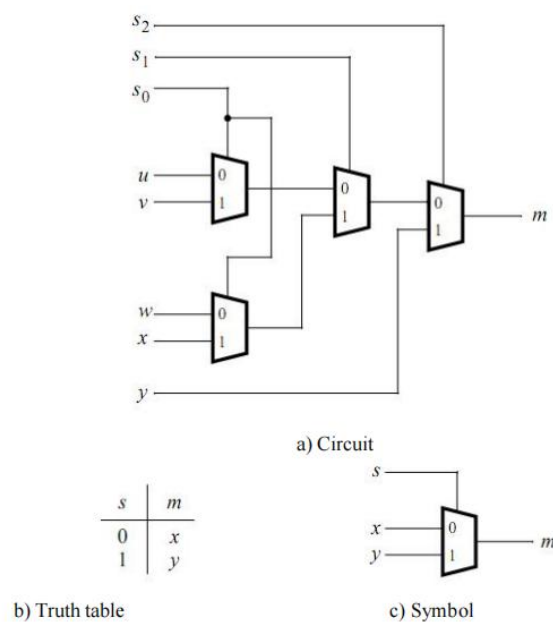


Figure 59. A 5-to-1 multiplexer.

Perform the following steps to implement the three-bit wide 5-to-1 multiplexer.

1. Create a new Quartus II project for your circuit.
2. Create a VHDL entity for the three-bit wide 5-to-1 multiplexer. Connect its select inputs to switches SW17–15, and use the remaining 15 switches SW14–0 to provide the five 3-bit inputs U to Y . Connect the SW switches to the red lights LEDR and connect the output M to the green lights LEDG2–0.
3. Include in your project the required pin assignments for the DE2 board. Compile the project.
4. Download the compiled circuit into the FPGA chip. Test the functionality of the three-bit wide 5-to-1 multiplexer by toggling the switches and observing the LEDs. Ensure that each of the inputs U to Y can be properly selected as the output M .

(4) 7-segment decoder

Figure 61 shows a 7-segment decoder module that has the three-bit input $c_2c_1c_0$. This decoder produces seven outputs that are used to display a character on a 7-segment display. Table 1 lists the characters that should be displayed for each valuation of $c_2c_1c_0$. To keep the design simple, only four characters are included in the table (plus the 'blank' character, which is selected for codes 100 – 111).

The seven segments in the display are identified by the indices 0 to 6 shown in the figure. Each

segment is illuminated by driving it to the logic value 0. You are to write a VHDL entity that implements logic functions that represent circuits needed to activate each of the seven segments. Use only simple VHDL assignment statements in your code to specify each logic function using a Boolean expression.

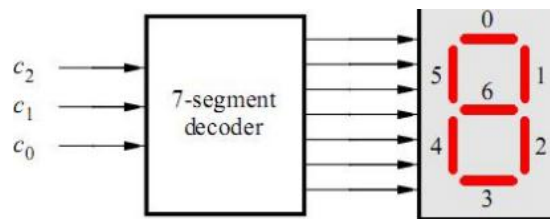


Figure 61. A 7-segment decoder.

$c_2 c_1 c_0$	Character
000	H
001	E
010	L
011	O
100	
101	
110	
111	

Table 1. Character codes.

Perform the following steps:

1. Create a new Quartus II project for your circuit.
2. Create a VHDL entity for the 7-segment decoder. Connect the $c_2 c_1 c_0$ inputs to switches SW2–0, and connect the outputs of the decoder to the HEX0 display on the DE2 board. The segments in this display are called HEX00, HEX01, ..., HEX06, corresponding to Figure 61. You should declare the 7-bit port `HEX0 : OUT STD LOGIC VECTOR(0 TO 6);` in your VHDL code so that the names of these outputs match the corresponding names in the DE2 User Manual and the DE2 pin assignments.csv file.
3. After making the required DE2 board pin assignments, compile the project.
4. Download the compiled circuit into the FPGA chip. Test the functionality of the circuit by toggling the SW2–0 switches and observing the 7-segment display.

(5)5-to-1 7-segment decoder

Consider the circuit shown in Figure 62. It uses a three-bit wide 5-to-1 multiplexer to enable the selection of five characters that are displayed on a 7-segment display. Using the 7-segment decoder from Part IV this circuit can display any of the characters H, E, L, O, and ‘blank’. The character codes are set according to Table 1 by using the switches SW14–0, and a specific character is selected for display by setting the switches SW17–15.

An outline of the VHDL code that represents this circuit is provided in Figure 63. Note that we have used the circuits from Parts III and IV as subcircuits in this code. You are to extend the code in Figure 63 so that it uses five 7-segment displays rather than just one. You will need to use five instances of each of the subcircuits. The purpose of your circuit is to display any word on the five displays that is composed of the characters in Table 1, and be able to rotate this word in a circular

fashion across the displays when the switches SW17–15 are toggled. As an example, if the displayed word is HELLO, then your circuit should produce the output patterns illustrated in Table 2.

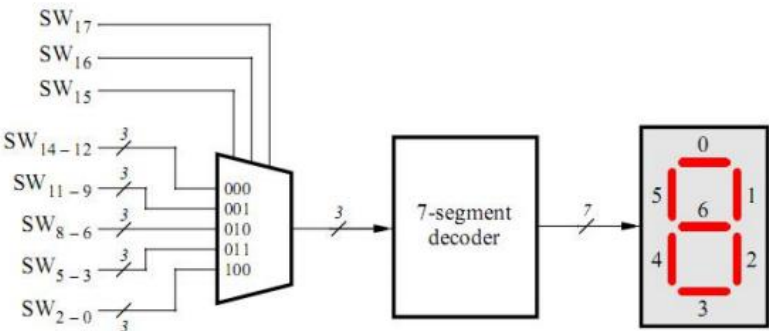


Figure 62. A circuit that can select and display one of five characters.

<i>SW</i> ₁₇ <i>SW</i> ₁₆ <i>SW</i> ₁₅	Character pattern				
000	H	E	L	L	O
001	E	L	L	O	H
010	L	L	O	H	E
011	L	O	H	E	L
100	O	H	E	L	L

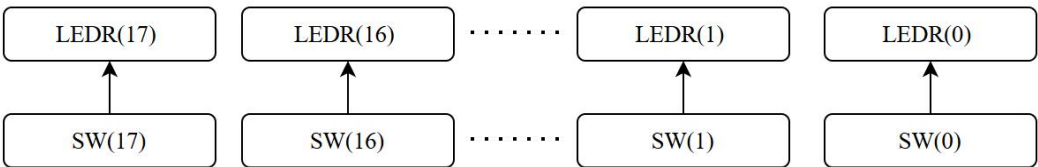
Table 2. Rotating the word HELLO on five displays.

Perform the following steps.

1. Create a new Quartus II project for your circuit.
2. Include your VHDL entity in the Quartus II project. Connect the switches SW17–15 to the select inputs of each of the five instances of the three-bit wide 5-to-1 multiplexers. Also connect SW14–0 to each instance of the multiplexers as required to produce the patterns of characters shown in Table 2. Connect the outputs of the five multiplexers to the 7-segment displays HEX4, HEX3, HEX2, HEX1, and HEX0.
3. Include the required pin assignments for the DE2 board for all switches, LEDs, and 7-segment displays. Compile the project.
4. Download the compiled circuit into the FPGA chip. Test the functionality of the circuit by setting the proper character codes on the switches SW14–0 and then toggling SW17–15 to observe the rotation of the characters.

2. Design Formulation

(1)switch display



Pic.1.1 design formulation of switch display

Both switches and LEDs are represented by 18-bit wide vectors where the index is decremented from 17 to zero. All bits of the input vector SW are copied into the output vector LEDR as Pic.1.1 using the simple signal assignment statement LEDR <= SW.

(2)2-to-1 multiplexer

The multiplexer can be described by the following VHDL statement:

$$m \leq (\text{NOT } (s) \text{ AND } x) \text{ OR } (s \text{ AND } y);$$

You are to write a VHDL entity that includes eight assignment statements like the one shown above to describe the circuit given in Figure 58a.

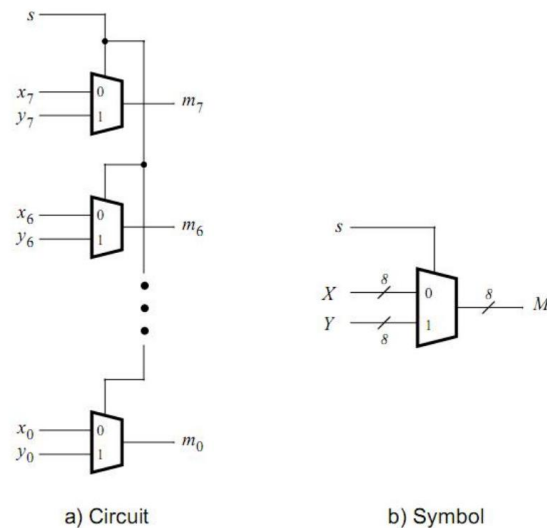
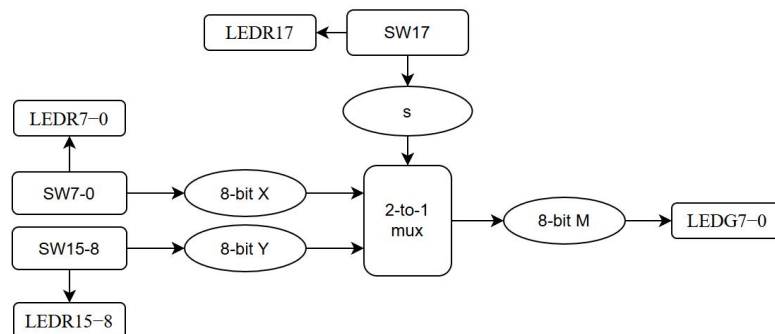


Figure 58. An eight-bit wide 2-to-1 multiplexer.

This circuit has two eight-bit inputs, X and Y, and produces the eight-bit output M. If $s=0$ then $M = X$, while if $s=1$ then $M = Y$. We refer to this circuit as an eight-bit wide 2-to-1 multiplexer. It has the circuit symbol shown in Figure 58b, in which X, Y, and M are depicted as eight-bit wires. The wiring structure can be seen in Pic.2.1.



Pic.2.1 design formulation of 2-to-1 multiplexer

(3)5-to-1 multiplexer

Recall from Figure 58 that an eight-bit wide 2-to-1 multiplexer can be built by using eight instances of a 2-to-1 multiplexer. Figure 60 applies this concept to define a three-bit wide 5-to-1 multiplexer. It contains three instances of the circuit in Figure 59a. The wiring structure can be seen in Pic.3.1.

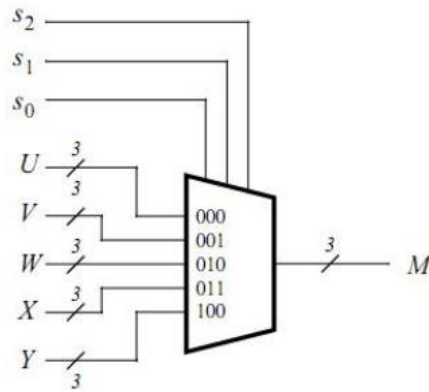
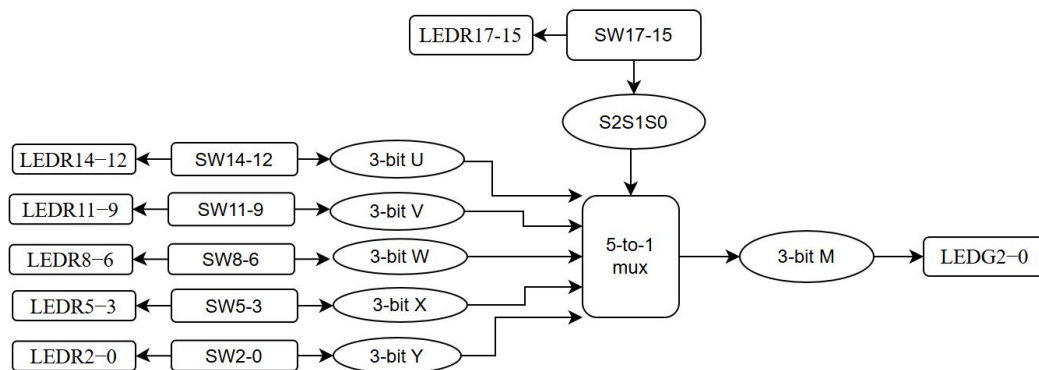


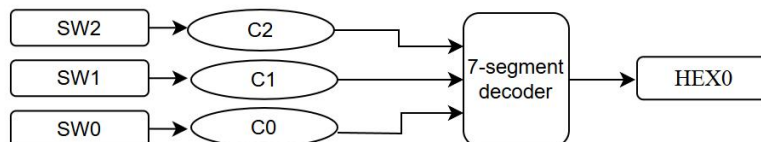
Figure 60. A three-bit wide 5-to-1 multiplexer.



Pic.3.1 design formulation of 5-to-1 multiplexer

(4) 7-segment decoder

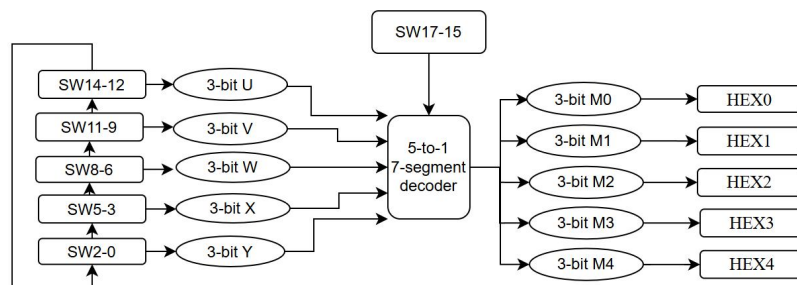
The wiring structure can be seen in Pic.4.1.



Pic.4.1 design formulation of 7-segment decoder

(5) 5-to-1 7-segment decoder

The wiring structure can be seen in Pic.5.1.



Pic.5.1 design formulation of 5-to-1 7-segment decoder

3. Design Entry

(1) switch display

switch_to_led.vhd:

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4  ENTITY switch_to_led IS
5  PORT (
6      SW : IN STD_LOGIC_VECTOR(17 DOWNTO 0); -- Input switches
7      LEDR : OUT STD_LOGIC_VECTOR(17 DOWNTO 0) -- Output LEDs
8  );
9  END switch_to_led;
10
11 ARCHITECTURE Behavior OF switch_to_led IS
12 BEGIN
13     LEDR <= SW; -- Connect all switches to their respective LEDs
14 END Behavior;
```

(2) 2-to-1 multiplexer

two_to_one_mux.vhd:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity two_to_one_mux is
5  Port ( s : in STD_LOGIC; -- Select input
6        X : in STD_LOGIC_VECTOR (7 downto 0); -- 8-bit input X
7        Y : in STD_LOGIC_VECTOR (7 downto 0); -- 8-bit input Y
8        M : out STD_LOGIC_VECTOR (7 downto 0) -- 8-bit output M
9  );
10 end two_to_one_mux;
11
12 architecture Behavioral of two_to_one_mux is
13 begin
14     -- 8-bit wide 2-to-1 multiplexer
15     M(0) <= (NOT s AND X(0)) OR (s AND Y(0));
16     M(1) <= (NOT s AND X(1)) OR (s AND Y(1));
17     M(2) <= (NOT s AND X(2)) OR (s AND Y(2));
18     M(3) <= (NOT s AND X(3)) OR (s AND Y(3));
19     M(4) <= (NOT s AND X(4)) OR (s AND Y(4));
20     M(5) <= (NOT s AND X(5)) OR (s AND Y(5));
21     M(6) <= (NOT s AND X(6)) OR (s AND Y(6));
22     M(7) <= (NOT s AND X(7)) OR (s AND Y(7));
23 end Behavioral;
```

two_to_one_mux_display.vhd:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity two_to_one_mux_display is
5  Port ( SW17 : in STD_LOGIC; -- Select input
6        SW7_0 : in STD_LOGIC_VECTOR (7 downto 0); -- 8-bit input X
7        SW15_8 : in STD_LOGIC_VECTOR (7 downto 0); -- 8-bit input Y
8        LEDR7_0 : out STD_LOGIC_VECTOR (7 downto 0); -- 8-bit input X for display
9        LEDR15_8 : out STD_LOGIC_VECTOR (7 downto 0); -- 8-bit input Y for display
10        LEDG7_0 : out STD_LOGIC_VECTOR (7 downto 0) -- 8-bit output M for display
11  );
12 end two_to_one_mux_display;
```

```

14 architecture Behavioral of two_to_one_mux_display is
15     component two_to_one_mux
16     Port ( s : in STD_LOGIC;
17           X : in STD_LOGIC_VECTOR (7 downto 0);
18           Y : in STD_LOGIC_VECTOR (7 downto 0);
19           M : out STD_LOGIC_VECTOR (7 downto 0)
20           );
21     end component;
22
23     signal M : STD_LOGIC_VECTOR (7 downto 0);
24
25 begin
26     -- Instantiate the 2-to-1 multiplexer
27     two_to_one_mux_inst:
28     two_to_one_mux port map (
29         s => SW17,
30         X => SW7_0,
31         Y => SW15_8,
32         M => M
33     );
34
35     -- Connect inputs to red LEDs for display
36     LEDR7_0 <= SW7_0;
37     LEDR15_8 <= SW15_8;
38
39     -- Connect output to green LEDs for display
40     LEDG7_0 <= M;
41 end Behavioral;

```

(3)5-to-1 multiplexer

five_to_one_mux_display.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity five_to_one_mux_display is
5  Port ( S2, S1, S0 : in STD_LOGIC; -- Selection lines
6        U, V, W, X, Y : in STD_LOGIC_VECTOR (2 downto 0); -- Data inputs
7        M : out STD_LOGIC_VECTOR (2 downto 0);
8        LEDR : out STD_LOGIC_VECTOR (17 downto 0)); -- Output data
9  end five_to_one_mux_display;
10
11 architecture Behavioral of five_to_one_mux_display is
12     signal s:STD_LOGIC_VECTOR (2 downto 0);
13 begin
14     process(S2, S1, S0, U, V, W, X, Y)
15     begin
16         s<=S2 & S1 & S0;
17         case s is
18             when "000" => M <= U;
19             when "001" => M <= V;
20             when "010" => M <= W;
21             when "011" => M <= X;
22             when "100" => M <= Y;
23             when others=> M <= "000";
24         end case;
25     end process;
26
27     -- Connect inputs to red LEDs for display
28     LEDR(17) <= S2;
29     LEDR(16) <= S1;
30     LEDR(15) <= S0;
31     LEDR(14) <= U(2);
32     LEDR(13) <= U(1);
33     LEDR(12) <= U(0);
34     LEDR(11) <= V(2);
35     LEDR(10) <= V(1);
36     LEDR(9) <= V(0);
37     LEDR(8) <= W(2);
38     LEDR(7) <= W(1);
39     LEDR(6) <= W(0);
40     LEDR(5) <= X(2);
41     LEDR(4) <= X(1);
42     LEDR(3) <= X(0);
43     LEDR(2) <= Y(2);
44     LEDR(1) <= Y(1);
45     LEDR(0) <= Y(0);
46 end Behavioral;

```

(4)7-segment decoder

seven-segment decoder.vhd:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity seven_segment_decoder is
5  Port ( C2, C1, C0 : in STD_LOGIC;
6        HEX0 : out STD_LOGIC_VECTOR (6 downto 0));
7  end seven_segment_decoder;
8
9  architecture Behavioral of seven_segment_decoder is
10     signal c :STD_LOGIC_VECTOR (2 downto 0);
11 begin
12     process(C2, C1, C0)
13     begin
14         c <= C2 & C1 & C0;
15         case c is
16             when "000" => HEX0 <= "0001001"; -- H
17             when "001" => HEX0 <= "0000110"; -- E
18             when "010" => HEX0 <= "1000111"; -- L
19             when "011" => HEX0 <= "1000000"; -- O
20             when others=> HEX0 <= "1111111"; -- Blank
21         end case;
22     end process;
23 end Behavioral;
```

(5)5-to-1 7-segment decoder

five_to_one_mux.vhd:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity five_to_one_mux is
5  Port ( S: in STD_LOGIC_VECTOR(2 downto 0); -- Selection lines
6        U, V, W, X, Y : in STD_LOGIC_VECTOR (2 downto 0); -- Data inputs
7        M : out STD_LOGIC_VECTOR (2 downto 0);
8        LEDR : out STD_LOGIC_VECTOR (17 downto 0)); -- Output data
9  end five_to_one_mux;
10
11 architecture Behavioral of five_to_one_mux is
12     signal s:STD_LOGIC_VECTOR (2 downto 0);
13 begin
14     process(S, U, V, W, X, Y)
15     begin
16         case S is
17             when "000" => M <= U;
18             when "001" => M <= V;
19             when "010" => M <= W;
20             when "011" => M <= X;
21             when "100" => M <= Y;
22             when others=> M <= "111";
23         end case;
24     end process;
25
26 end Behavioral;
```

seven_segment_decoder.vhd:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity seven_segment_decoder is
5  Port ( C : in STD_LOGIC_VECTOR(2 downto 0);
6        Display : out STD_LOGIC_VECTOR (6 downto 0));
7  end seven_segment_decoder;
```

```

9  architecture Behavioral of seven_segment_decoder is
10 begin
11     process(C)
12     begin
13         case C is
14             when "000" => Display <= "0001001"; -- H
15             when "001" => Display <= "0000110"; -- E
16             when "010" => Display <= "1000111"; -- L
17             when "011" => Display <= "1000000"; -- O
18             when others=> Display <= "1111111"; -- Blank
19         end case;
20     end process;
21 end Behavioral;

```

five_to_one_7segment_decoder.vhd:

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4  ENTITY five_to_one_7segment_decoder IS
5  PORT ( SW : IN STD_LOGIC_VECTOR(17 DOWNTO 0);
6        shift_reg : buffer STD_LOGIC_VECTOR(14 DOWNTO 0);
7        M0,M1,M2,M3,M4 : buffer STD_LOGIC_VECTOR(2 DOWNTO 0);
8        HEX0, HEX1, HEX2, HEX3, HEX4 : OUT STD_LOGIC_VECTOR(6 DOWNTO 0));
9  END five_to_one_7segment_decoder;
10
11 ARCHITECTURE Behavior OF five_to_one_7segment_decoder IS
12     COMPONENT five_to_one_mux
13     PORT (S, U, V, W, X, Y : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
14          M : OUT STD_LOGIC_VECTOR(2 DOWNTO 0));
15     END COMPONENT;
16
17     COMPONENT seven_segment_decoder
18     PORT (C : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
19          Display : OUT STD_LOGIC_VECTOR(6 DOWNTO 0));
20     END COMPONENT;
21
22 BEGIN
23     shift_reg <= SW(14 DOWNTO 0);
24     -- Instantiate the multiplexers
25     MUX0: five_to_one_mux PORT MAP (SW(17 DOWNTO 15), shift_reg(14 DOWNTO 12), shift_reg(11 DOWNTO 9),
26                                     shift_reg(8 DOWNTO 6), shift_reg(5 DOWNTO 3), shift_reg(2 DOWNTO 0), M0);
27     MUX1: five_to_one_mux PORT MAP (SW(17 DOWNTO 15), shift_reg(11 DOWNTO 9), shift_reg(8 DOWNTO 6),
28                                     shift_reg(5 DOWNTO 3), shift_reg(2 DOWNTO 0), shift_reg(14 DOWNTO 12), M1);
29     MUX2: five_to_one_mux PORT MAP (SW(17 DOWNTO 15), shift_reg(8 DOWNTO 6), shift_reg(5 DOWNTO 3),
30                                     shift_reg(2 DOWNTO 0), shift_reg(14 DOWNTO 12), shift_reg(11 DOWNTO 9), M2);
31     MUX3: five_to_one_mux PORT MAP (SW(17 DOWNTO 15), shift_reg(5 DOWNTO 3), shift_reg(2 DOWNTO 0),
32                                     shift_reg(14 DOWNTO 12), shift_reg(11 DOWNTO 9), shift_reg(8 DOWNTO 6), M3);
33     MUX4: five_to_one_mux PORT MAP (SW(17 DOWNTO 15), shift_reg(2 DOWNTO 0), shift_reg(14 DOWNTO 12),
34                                     shift_reg(11 DOWNTO 9), shift_reg(8 DOWNTO 6), shift_reg(5 DOWNTO 3), M4);
35
36     -- Instantiate the decoders
37     CHAR0: seven_segment_decoder PORT MAP (M0, HEX0);
38     CHAR1: seven_segment_decoder PORT MAP (M1, HEX1);
39     CHAR2: seven_segment_decoder PORT MAP (M2, HEX2);
40     CHAR3: seven_segment_decoder PORT MAP (M3, HEX3);
41     CHAR4: seven_segment_decoder PORT MAP (M4, HEX4);
42 END Behavior;

```

4. Simulation and Synthesis Results

(1)switch display

Master Time		19.05 ns		Pointer:	965.78 us	Interval:	965.76 us	Start:	End																		
	Name	V...	1...		965.4 us	965.56 us	965.72 us	965.88 us	966.04 us	966.2 us	966.36 us	966.52 us															
	SW	H...			1022C	1023D	1024E	1025F	10270	10281	10292	102A3	102B4	102C5	102D6	102E7	102F8	10309	1031A	1032B	1033C	1034D	1035E	1036F	10380	10391	03A2
	LEDR	H...			1022C	1023D	1024E	1025F	10270	10281	10292	102A3	102B4	102C5	102D6	102E7	102F8	10309	1031A	1032B	1033C	1034D	1035E	1036F	10380	10391	03A2

Master Time		19.05 ns	Pointer:	26.75 us	Interval:	26.73 us	Start:	End:												
	Name	V...	26.18 us	26.26 us	26.34 us	26.42 us	26.5 us	26.58 us	26.66 us	26.74 us	26.82 us	26.9 us	26.98 us							
0	X	H 00	0B	0C	0D	0E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C
1	Y	H 00	EE	CC	DD	EE	FF	10	21	32	43	54	65	76	87	98	A9	BA	CB	DC
18	s	B 0																		
19	M	H 00	BB	0C	DD	0E	FF	10	21	12	43	14	65	16	87	18	A9	1A	CB	1C

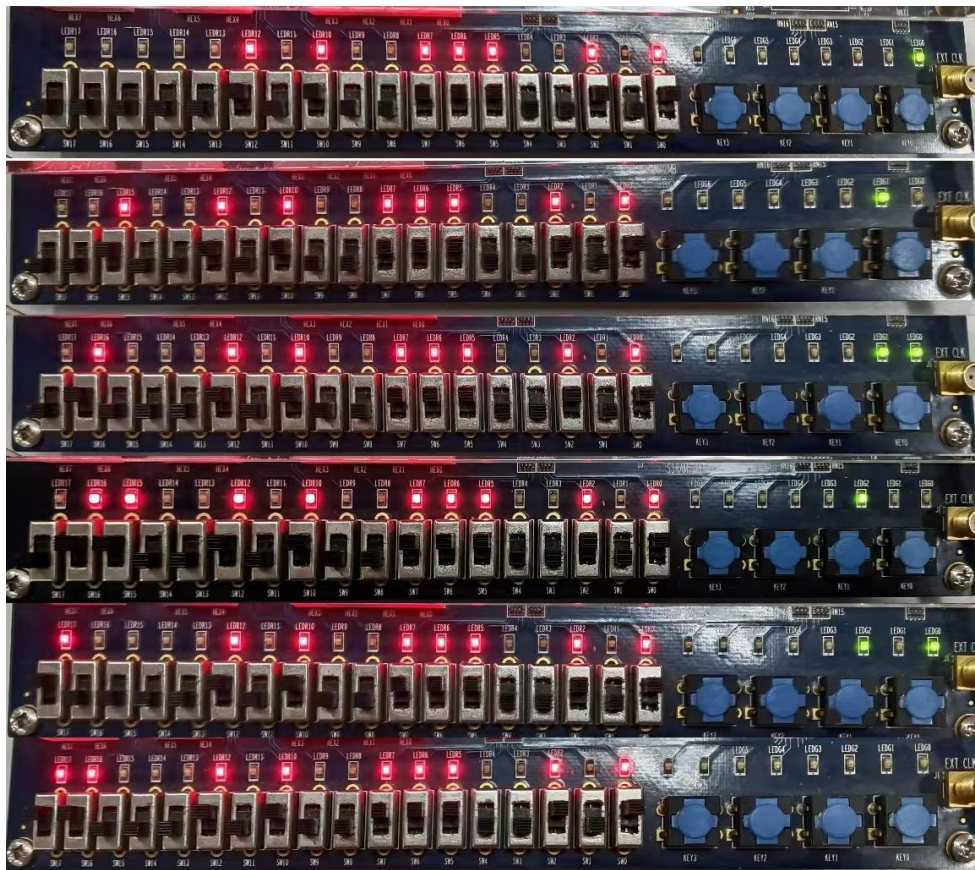
Timing diagram showing Master Time, Pointer, and various signals (S0, S1, S2, U, V, W, X, Y, M) over time. The diagram includes a timeline with markers for 19.05 ns, 6.23 us, 6.21 us, 6.38 us, 6.46 us, 6.54 us, and 6.62 us. The signals are represented by horizontal bars indicating their active periods.

Timing diagram showing Master Time, Pointer, Interval, Start, and End signals. The diagram includes a timeline from 0 ps to 480.0 ns with a 40.0 ns scale. Signals C2, C1, C0, and C3 are shown as digital waveforms. A data bus is shown at the bottom with hexadecimal values: 0001001, 0000110, 1000111, 1000000, 1111111, 0001001, and 0000110.

PC	Op	SW	B	0000000001	000101001	0000101001	0000101001	0000101001	0000101001	0000101001	0000101001	0000101001	0000101001	0000101001	0000101001	0000101001	000101001	000101001
19	HEX0	H C	09	06	47	40	7F	09	06	47	40	7F						
27	HEX1	H C	06	47	47	40	09	7F	06	47	47	40	09	7F				
35	HEX2	H 4	47	47	40	09	06	7F	47	47	40	09	06	7F				
43	HEX3	H 4	47	40	09	06	47	7F	47	40	09	06	47	7F				
51	HEX4	H 4	40	09	06	47	47	7F	40	09	06	47	47	7F				
59	shift_reg	H 02	0293															
75	M0	H 1	0	1	2	3	7	0	1	2	3	7						
79	M1	H	1	2	2	3	0	7	1	2	2	3	0	7				
83	M2	H:	2	2	3	0	1	7	2	2	3	0	1	7				
87	M3	H:	2	3	0	1	2	7	2	3	0	1	2	7				
91	M4	H:	3	0	1	2	2	7	3	0	1	2	2	7				

(3) and (5) are proved in experiment on board.

S2S1S0, U, V, W, X, Y and Z are lined from left to right and red LED light lights up when the switch is pushed up. In the experiment, U, V, W, X, Y and Z are relatively assigned as 1-5, and results are shown as lit green LED. In Pic.3.2, when S2S1S0 is assigned from 000 to 100, M is programmed as 001 to 101; when S2S1S0 is assigned as 110, M is assigned as 000 and all green LED are not lit.

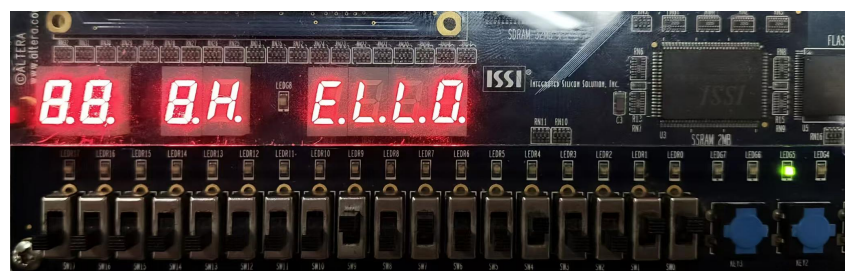


Pic.3.2 5-to-1 multiplexer

(5)5-to-1 7-segment decoder

S is controlled by SW17-15 and valid input is 000-100. S is initially set 000 and push SW14-0 so that HEX4-HEX0 display word “HELLO”. When S is added by 001, word on display is shifted to the left by one character, and results are shown in Pic.5.2-Pic.5.6.

In Pic.5.2 S is assigned as 000, and digital tube demonstrates word “HELLO”.



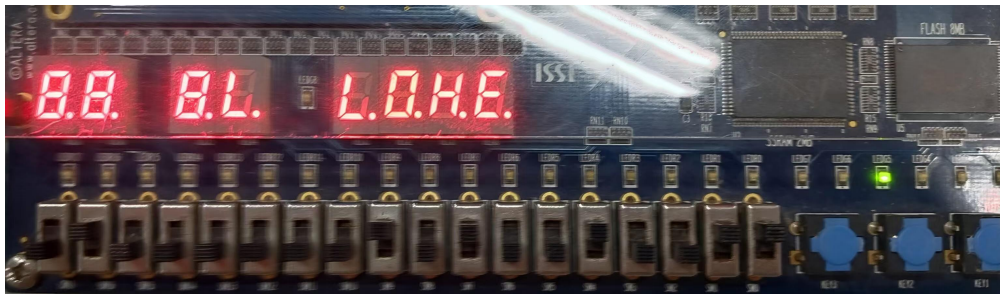
Pic.5.2 5-to-1 7-segment decoder(000=>HELLO)

In Pic.5.3 S is assigned as 001, and digital tube demonstrates word “ELLOH”.



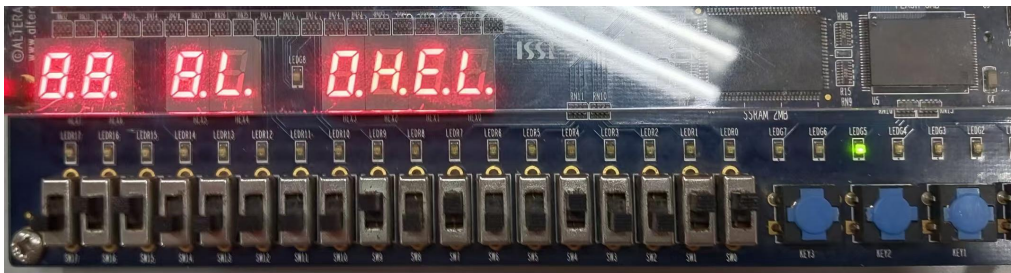
Pic.5.3 5-to-1 7-segment decoder(001=ELLOH)

In Pic.5.4 S is assigned as 010, and digital tube demonstrates word “LLOHE”.



Pic.5.4 5-to-1 7-segment decoder(010=LLOHE)

In Pic.5.5 S is assigned as 011, and digital tube demonstrates word “LOHEL”.



Pic.5.5 5-to-1 7-segment decoder(011=LOHEL)

In Pic.5.6 S is assigned as 100, and digital tube demonstrates word “OHELL”.



Pic.5.6 5-to-1 7-segment decoder(100=OHELL)

6. Discussion and Conclusion

When programming 5-to-1 7-segment decoder in (5), I firstly put SW14-0 in an additional array and shift it to left with every add. But actually 5-to-1 multiplexer has already does the shifting job when choosing the number. Thus there is no need to call on another shifting module. Also, in emulation I put HELLO relatively into HEX0-HEX4, but experiment on board turned out that HEX0-HEX4 are line from right to left, so in physical experiment I reversed sequence of the output M0-M4 and result ended up correct.

Building such a basic multiplexer helps to understand the basic concepts of digital logic and also to see how the theoretical knowledge can be implemented in real hardware. As the complexity of the design increases, I learned how to decompose a problem into smaller sub-problems and combine them to solve the problem. It also shows how the behavior of the output can be changed by adjusting the inputs to achieve a dynamic display. Throughout, I not only improved my understanding of the VHDL language, but also gained valuable hands-on experience that will be very helpful for future careers in the field of digital system design. In addition, through these practices, I have developed my ability to analyze problems and design solutions, which are extremely important skills in the engineering field.